

csiro-biomass

13th Place Solution — Frozen DINOv3-7B + Stacked Ridge

Rank	13
Team	Forrest_xlz
Author	Forrest_xlz
Topic ID	671818
Version	Original

Source: <https://www.kaggle.com/competitions/csiro-biomass/discussion/671818>

Retrieved with Kaggle CLI on 2026-07-18.

Thanks to the competition organizers for hosing this challenging competition. Thanks to [@zjayzz](#) for the [public notebook](#) — your work gave me a great perspective and a solid baseline to complete this task. This is my first time achieving such a great result on kaggle. I have learned a lot from this competition, and I will introduce the methods I used below:

Overview

Because the training set in CSIRO Biomass is extremely small, I intentionally followed a “frozen foundation model + lightweight supervised head” recipe to reduce overfitting and keep training cost negligible. This is well aligned with DINOv3’s downstream practice: the paper shows that a fully frozen DINOv3 backbone can remain highly competitive when paired with a relatively small task-specific adapter/decoder, which is particularly attractive for small-data regimes and efficient multi-task reuse.

The pasture biomass prediction task sits between global recognition and dense understanding: local regions (coverage, dryness, shadows, texture) contribute unequally to the final biomass targets. DINOv3 is reported to be consistently strong across both classification and dense prediction benchmarks, with notably improved local features on dense tasks—exactly the type of representation we want for “local evidence aggregated into global regression.”

Concretely, I kept DINOv3 frozen as a feature extractor and built a simple yet effective pipeline around multi-view representations + stacked Ridge regressors: multi-scale (256/512/768) left-right views capture local variations, while multi-depth token taps (last layer + selected intermediate blocks) exploit both high-level semantics and more geometry/structure-oriented mid-level features. DINOv3’s per-layer analysis suggests that intermediate layers can be beneficial for geometry-heavy tasks, motivating our mid-layer taps.

I also found multi-resolution ensembling helpful, consistent with the paper’s emphasis on high-resolution adaptation and multi-resolution inference for better local features.

Finally, under this frozen + Ridge stacking setup, scaling the backbone (Large → Huge → 7B) improved performance with minimal re-tuning, while Ridge training itself remained almost instantaneous.

Competition Understanding

How I Used Leaderboard Feedback to Refine My Understanding

In this competition, I tried to be very explicit about what information I had and how reliable it was.

My understanding of the problem came from three sources: the competition description and training data, my own prior knowledge, and the public leaderboard score. The first two were valuable, but they were also noisy. Any prior understanding—no matter whether it comes from intuition, experience, or related papers—inevitably contains bias.

So instead of treating my initial understanding as truth, I treated it as a prior belief that needed to be updated. In a Bayesian sense, the public leaderboard score served as an external signal to revise that prior.

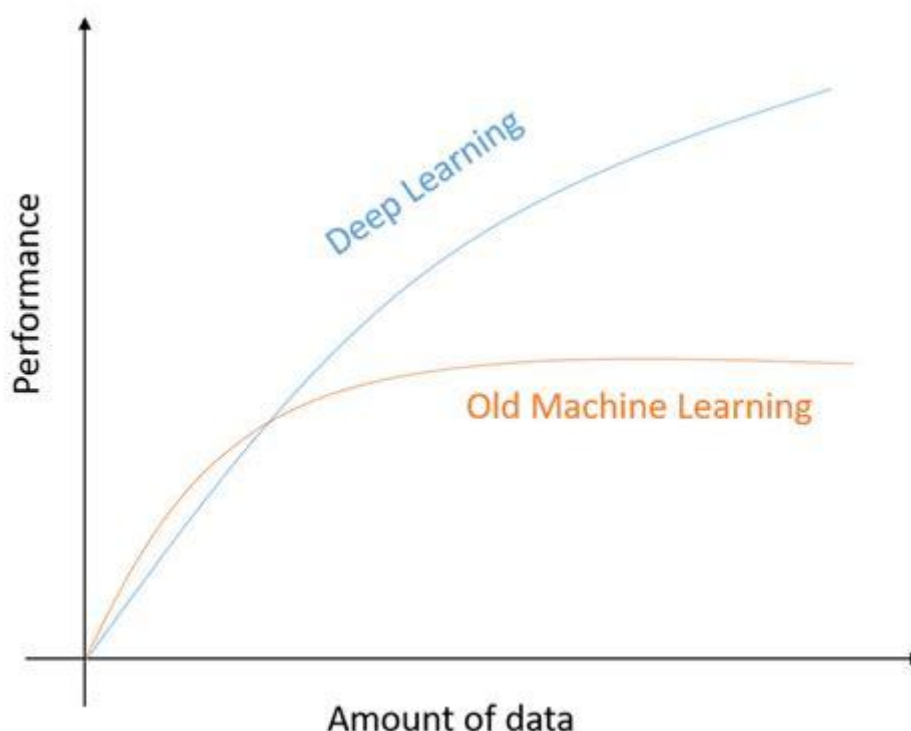
I often thought of the process as moving through a dark space with a torch. My intuition gave me an initial direction, but only the leaderboard feedback could illuminate part of the landscape. Each experiment, whether based on my own code or reproduced public ideas, acted like a small torch that revealed another part of the problem. By combining these local observations, I gradually formed a better global understanding of the task.

This way of thinking helped me focus not only on model improvement, but also on understanding the structure of the competition itself: whether the small dataset favored traditional models, whether forage quality recognition driven more by global information or local patterns, and which assumptions were truly supported by empirical results.

In other words, I was not just searching for a better score. I was using score feedback to iteratively correct my understanding of the problem.

Why Chose Stack Traditional Models

Better Generalization on Small Data

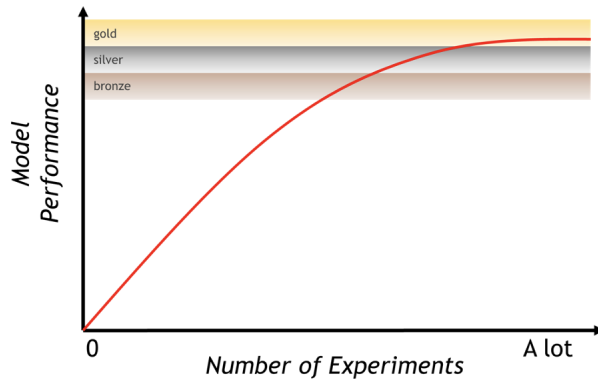


When the amount of data is small, traditional machine learning methods often generalize better than deep neural networks. In this competition, the training set contained only 357 images, which is relatively small.

At the beginning of the competition, I tried many approaches based on deep neural networks, but I found a large gap between my CV score and public leaderboard score. Later, I switched to stacking traditional models such as Ridge, XGBoost, and LightGBM. With this approach, I observed a much more stable relationship

between CV and public score, and it became easier to improve leaderboard performance through further experimentation. That is why I finally chose this direction.

More Experiments Without Complex Hyperparameter Tuning



The number of experiments is crucial for achieving strong performance. This is something I learned from [this writeup](#) by [Chris Deotte](#).

If you only try a few ideas, it is difficult to reach top performance. Compared with deep neural networks, traditional models require much less low-level hyperparameter tuning and train much faster. This allowed me to spend more time iterating on higher-level design choices, such as feature extraction, backbone selection, and ensembling, instead of getting stuck tuning details like learning rate, batch size, and number of epochs.

Easy to Scale With Better Backbones

When I switched to a stronger backbone model (Base → Large → Huge → 7B), I could often achieve higher scores without restarting the whole training process from scratch.

This made the pipeline much easier to scale. Once I had good features from a better backbone, I could quickly test them with the same downstream traditional models.

Ridge vs. XGBoost vs. LightGBM

Model

Architecture

```
Input → Views
├
├ Left-right split → {L, R}
├ Scales S = {256, 512, 768}
```

```

Frozen backbone
├─ DINOv3 ViT-7B (BF16 inference, final norm)
├─ token tap L0: last layer
├─ token taps L1,L2: mid blocks {35, 38}
→ Layers L = {last, b35, b38}

View definition (9 views)
├─ Each view is a (scale, layer) pair:  $v = (s, \ell)$ , with  $s \in S$  and  $\ell \in L$ 
→ 3 scales  $\times$  3 taps = 9 views total
(e.g.,  $v1=(256, \text{last})$ ,  $v2=(256, b35)$ , ...,  $v9=(768, b38)$ )

Per-view embedding (computed independently for each  $v$ )
├─ Patch tokens (drop CLS/prefix) → pool over patches: mean + max + min + signed-GeM(p=3)
→ embedding matrix  $E_v$ :  $[n_{\text{samples}}, d_v]$  where  $d_v = 4C$ 
→ RobustScaler fitted/applied per view (per  $E_v$ )

Stacking (Ridge-only)
├─ Level-1: one base model per view
├─ For each view  $v=1..9$ :
├─ MultiOutput Ridge( $\alpha=5$ ):  $E_v \rightarrow$  OOF predictions  $P_v$ 
├─ where  $P_v$  is  $[n_{\text{train\_samples}}, 5]$  (5 biomass targets)
├─ Level-2: meta model per target
├─ For each target  $t \in \{1..5\}$ :
 $X_{\text{meta}}(t) = \text{concat of } \{P_v[:, t]\} \text{ over } v=1..9 \rightarrow [n_{\text{train\_samples}}, 9]$ 
Meta Ridge( $\alpha=1$ ):  $X_{\text{meta}}(t) \rightarrow \hat{y}_t$ 

Dual label-space ensemble (outside meta)
├─ Run the whole stack on raw  $y$ 
├─ Run the whole stack on  $\log_{1p}(y) \rightarrow \text{expm1}$  back
├─ Blend:  $\hat{y} = 0.5 \cdot \hat{y}_{\text{raw}} + 0.5 \cdot \hat{y}_{\text{log}}$ 

Outer ensemble + constraints
├─ Repeat with  $n_{\text{folds}} \in \{2, 3, 5\} \rightarrow$  average
├─ Physics post-process:  $\text{clip} \geq 0$ ;  $\text{GDM} = \text{Green} + \text{Clover}$ ;  $\text{Total} = \text{GDM} + \text{Dead}$ 

Output: [Green, Dead, Clover, GDM, Total] per sample

```

Here is my main model architecture; for more details, please refer to the notebook below.

Key Points

- **Multi-scale resolution** ensembling (256, 512, 768)
- **Intermediate-layer feature** taps (using mid-block tokens(35, 38))
- **multi-pooling** aggregation (mean / max / min / signed-GeM)
- **Raw-space + log-space** ensembling (train twice, blend in raw space)
- **High-resolution** feature extraction / inference
- Strong inductive bias from heavily regularized **Ridge regression**

- **Multi-fold** ensembling (repeat with different KFold settings and average)
-

Ablation Study

ablation / variant	public score	private score
base	0.75696	0.64833
multi-scale (768 only)	0.75332	0.64324
multi-scale (256, 512)	0.74585	0.63738
without mid layers	0.74969	0.62957
mid layer (35 only)	0.75466	0.64466
mid layer (38 only)	0.75218	0.62950
ensemble folds (5 only)	0.75493	0.64774
without post process	0.75208	0.64986
only raw-space	0.74358	0.64147
only avg pooling	0.72056	0.60662

What Didn't Work

- **Stacked XGBoost** — I tried a simple setup, but it didn't improve performance.
-

Future work

From the ablation study above, the two most impactful components are the mid-layer features and the feature pooling strategy. Both directly affect the quality of the extracted representations, so future work can focus on improving this part of the pipeline—for example, exploring stronger feature aggregation modules such as attention-based pooling.

Reference

[Dinov3](#)

[Stacked Generalization: An Introduction to Super Learning](#)

Stacked Generalization: when does it work?

GeM pooling

Below is my code, which runs both training and inference end-to-end, along with the DINOv3-7B model used in this solution.